



14.2 输入和输出运算符

如我们所知，IO 标准库分别使用>>>和<<执行输入和输出操作。对于这两个运算符来说，IO 库定义了用其读写内置类型的版本，而类则需要自定义适合其对象的新版本以支持 IO 操作。



14.2.1 重载输出运算符<<

通常情况下，输出运算符的第一个形参是一个非常量 ostream 对象的引用。之所以 ostream 是非常量是因为向流写入内容会改变其状态；而该形参是引用是因为我们无法直接复制一个 ostream 对象。

第二个形参一般来说是一个常量的引用，该常量是我们想要打印的类类型。第二个形参是引用的原因是我们希望避免复制实参；而之所以该形参可以是常量是因为（通常情况下）打印对象不会改变对象的内容。

为了与其他输出运算符保持一致，operator<<一般要返回它的 ostream 形参。

Sales_data 的输出运算符

举个例子，我们按照如下形式编写 Sales_data 的输出运算符：

```
ostream &operator<<(ostream &os, const Sales_data &item)
{
    os << item.isbn() << " " << item.units_sold << " "
        << item.revenue << " " << item.avg_price();
    return os;
}
```

除了名字之外，这个函数与之前的 print 函数（参见 7.1.3 节，第 234 页）完全一样。打印一个 Sales_data 对象意味着要分别打印它的三个数据成员以及通过计算得到的平均销售价格，每个元素以空格隔开。完成输出后，运算符返回刚刚使用的 ostream 的引用。

输出运算符尽量减少格式化操作

用于内置类型的输出运算符不太考虑格式化操作，尤其不会打印换行符，用户希望类的输出运算符也像如此行事。如果运算符打印了换行符，则用户就无法在对象的同一行内接着打印一些描述性的文本了。相反，令输出运算符尽量减少格式化操作可以使用户有权控制输出的细节。

Best
Practices

通常，输出运算符应该主要负责打印对象的内容而非控制格式，输出运算符不应该打印换行符。

输入输出运算符必须是非成员函数

与 istream 标准库兼容的输入输出运算符必须是普通的非成员函数，而不能是类的成员函数。否则，它们的左侧运算对象将是我们的类的一个对象：

```
Sales_data data;
data << cout;           // 如果 operator<<是 Sales_data 的成员
```

假设输入输出运算符是某个类的成员，则它们也必须是 istream 或 ostream 的成员。然而，这两个类属于标准库，并且我们无法给标准库中的类添加任何成员。

因此, 如果我们希望为类自定义 IO 运算符, 则必须将其定义成非成员函数。当然, IO 运算符通常需要读写类的非公有数据成员, 所以 IO 运算符一般被声明为友元(参见 7.2.1 节, 第 241 页)。

558

14.2.1 节练习

练习 14.6: 为你的 `Sales_data` 类定义输出运算符。

练习 14.7: 你在 13.5 节的练习(第 470 页)中曾经编写了一个 `String` 类, 为它定义一个输出运算符。

练习 14.8: 你在 7.5.1 节的练习 7.40(第 261 页)中曾经选择并编写了一个类, 为它定义一个输出运算符。

14.2.2 重载输入运算符>>



通常情况下, 输入运算符的第一个形参是运算符将要读取的流的引用, 第二个形参是将要读入到的(非常量)对象的引用。该运算符通常会返回某个给定流的引用。第二个形参之所以必须是个非常量是因为输入运算符本身的目的就是将数据读入到这个对象中。

`Sales_data` 的输入运算符

举个例子, 我们将按照如下形式编写 `Sales_data` 的输入运算符:

```
istream &operator>>(istream &is, Sales_data &item)
{
    double price; // 不需要初始化, 因为我们将先读入数据到 price, 之后才使用它
    is >> item.bookNo >> item.units_sold >> price;
    if (is) // 检查输入是否成功
        item.revenue = item.units_sold * price;
    else
        item = Sales_data(); // 输入失败: 对象被赋予默认的状态
    return is;
}
```

除了 `if` 语句之外, 这个定义与之前的 `read` 函数(参见 7.1.3 节, 第 234 页)完全一样。`if` 语句检查读取操作是否成功, 如果发生了 IO 错误, 则运算符将给定的对象重置为空 `Sales_data`, 这样可以确保对象处于正确的状态。

Note

输入运算符必须处理输入可能失败的情况, 而输出运算符不需要。

输入时的错误

559

在执行输入运算符时可能发生下列错误:

- 当流含有错误类型的数据时读取操作可能失败。例如在读取完 `bookNo` 后, 输入运算符假定接下来读入的是两个数字数据, 一旦输入的不是数字数据, 则读取操作及后续对流的其他使用都将失败。
- 当读取操作到达文件末尾或者遇到输入流的其他错误时也会失败。

在程序中我们没有逐个检查每个读取操作, 而是等读取了所有数据后赶在使用这些数据前一次性检查:

```

if (is)                                // 检查输入是否成功
    item.revenue = item.units_sold * price;
else
    item = Sales_data();               // 输入失败：对象被赋予默认的状态

```

如果读取操作失败，则 `price` 的值将是未定义的。因此，在使用 `price` 前我们需要首先检查输入流的合法性，然后才能执行计算并将结果存入 `revenue`。如果发生了错误，我们无须在意到底是哪部分输入失败，只要将一个新的默认初始化的 `Sales_data` 对象赋予 `item` 从而将其重置为空 `Sales_data` 就可以了。执行这样的赋值后，`item` 的 `bookNo` 成员将是一个空 `string`，`revenue` 和 `units_sold` 成员将等于 0。

如果在发生错误前对象已经有一部分被改变，则适时地将对象置为合法状态显得异常重要。例如在这个输入运算符中，我们可能在成功读取新的 `bookNo` 后遇到错误，这意味着对象的 `units_sold` 和 `revenue` 成员并没有改变，因此有可能会将这两个数据与一条完全不匹配的 `bookNo` 组合在一起。

通过将对象置为合法的状态，我们能（略微）保护使用者免于受到输入错误的影响。此时的对象处于可用状态，即它的成员都是被正确定义的。而且该对象也不会产生误导性的结果，因为它的数据在本质上确实是一体的。

Best
Practices

当读取操作发生错误时，输入运算符应该负责从错误中恢复。

标示错误

一些输入运算符需要做更多数据验证的工作。例如，我们的输入运算符可能需要检查 `bookNo` 是否符合规范的格式。在这样的例子中，即使从技术上来看 IO 是成功的，输入运算符也应该设置流的条件状态以标示出失败信息（参见 8.1.2 节，第 279 页）。通常情况下，输入运算符只设置 `failbit`。除此之外，设置 `eofbit` 表示文件耗尽，而设置 `badbit` 表示流被破坏。最好的方式是由 IO 标准库自己来标示这些错误。

560

14.2.2 节练习

练习 14.9：为你的 `Sales_data` 类定义输入运算符。

练习 14.10：对于 `Sales_data` 的输入运算符来说如果给定了下面的输入将发生什么情况？

(a) 0-201-999999-9 10 24.95 (b) 10 24.95 0-210-999999-9

练习 14.11：下面的 `Sales_data` 输入运算符存在错误吗？如果有，请指出来。对于这个输入运算符如果仍然给定上个练习的输入将发生什么情况？

```

istream& operator>>(istream& in, Sales_data& s)
{
    double price;
    in >> s.bookNo >> s.units_sold >> price;
    s.revenue = s.units_sold * price;
    return in;
}

```

练习 14.12：你在 7.5.1 节的练习 7.40（第 261 页）中曾经选择并编写了一个类，为它定义一个输入运算符并确保该运算符可以处理输入错误。

14.3 算术和关系运算符

通常情况下，我们把算术和关系运算符定义成非成员函数以允许对左侧或右侧的运算对象进行转换（参见 14.1 节，第 492 页）。因为这些运算符一般不需要改变运算对象的状态，所以形参都是常量的引用。

算术运算符通常会计算它的两个运算对象并得到一个新值，这个值有别于任意一个运算对象，常常位于一个局部变量之内，操作完成后返回该局部变量的副本作为其结果。如果类定义了算术运算符，则它一般也会定义一个对应的复合赋值运算符。此时，最有效的方式是使用复合赋值来定义算术运算符：

```
// 假设两个对象指向同一本书
Sales_data
operator+(const Sales_data &lsh, const Sales_data &rsh)
{
    Sales_data sum = lsh;           // 把 lsh 的数据成员拷贝给 sum
    sum += rsh;                     // 将 rsh 加到 sum 中
    return sum;
}
```

这个定义与原来的 add 函数（参见 7.1.3 节，第 234 页）是完全等价的。我们把 lsh 拷贝给局部变量 sum，然后使用 Sales_data 的复合赋值运算符（将在第 500 页定义）将 rsh 的值加到 sum 中，最后函数返回 sum 的副本。



如果类同时定义了算术运算符和相关的复合赋值运算符，则通常情况下应该使用复合赋值来实现算术运算符。

561

14.3 节练习

练习 14.13: 你认为 Sales_data 类还应该支持哪些其他算术运算符（参见表 4.1，第 124 页）？如果有的话，请给出它们的定义。

练习 14.14: 你觉得为什么调用 operator+= 来定义 operator+ 比其他方法更有效？

练习 14.15: 你在 7.5.1 节的练习 7.40（第 261 页）中曾经选择并编写了一个类，你认为它应该含有其他算术运算符吗？如果是，请实现它们；如果不是，解释原因。

14.3.1 相等运算符



通常情况下，C++ 中的类通过定义相等运算符来检验两个对象是否相等。也就是说，它们会比较对象的每一个数据成员，只有当所有对应的成员都相等时才认为两个对象相等。依据这一思想，我们的 Sales_data 类的相等运算符不但应该比较 bookNo，还应该比较具体的销售数据：

```
bool operator==(const Sales_data &lsh, const Sales_data &rsh)
{
    return lsh.isbn() == rsh.isbn() &&
           lsh.units_sold == rsh.units_sold &&
           lsh.revenue == rsh.revenue;
}

bool operator!=(const Sales_data &lsh, const Sales_data &rsh)
```